

Chargement d'un array depuis un fichier

Le module NumPy contient aussi des fonctions pour lire des données à partir de fichiers et créer des arrays automatiquement. C'est très pratique, car la plupart du temps les données que l'on analyse proviennent de fichiers. La fonction la plus simple à prendre en main est `np.loadtxt()`. Celle-ci lit un fichier organisé en lignes et colonnes. Par exemple, imaginons que nous ayons un fichier `donnees.txt` contenant :

```
1 7 310
15 -4 35
78 95 79
```

La fonction prend en argument le nom du fichier et renvoie un array 2D directement :

`np.loadtxt("donnees.txt")`

```
1 import numpy as np
2 A=np.loadtxt("donnees.txt")
-
```

```
1 A
```

```
array([[ 1.,  7., 310.],
       [15., -4.,  35.],
       [78., 95.,  79.]])
```

```
1 B=np.random.randint(10,100,(4,4))
2 np.savetxt("out.txt", B)
```

```
1 C=np.loadtxt("out.txt"); C
```

```
array([[39., 92., 32., 66.],
       [17., 94., 58., 95.],
       [97., 42., 86., 20.],
       [39., 77., 44., 55.]])
```

Les références partagées et le module copy

- Par erreur, il y a des personnes qui pensent que la manière la plus simple de copier une liste est de déclarer une variable et de lui affecter ladite liste comme dans l'exemple ci-dessous:

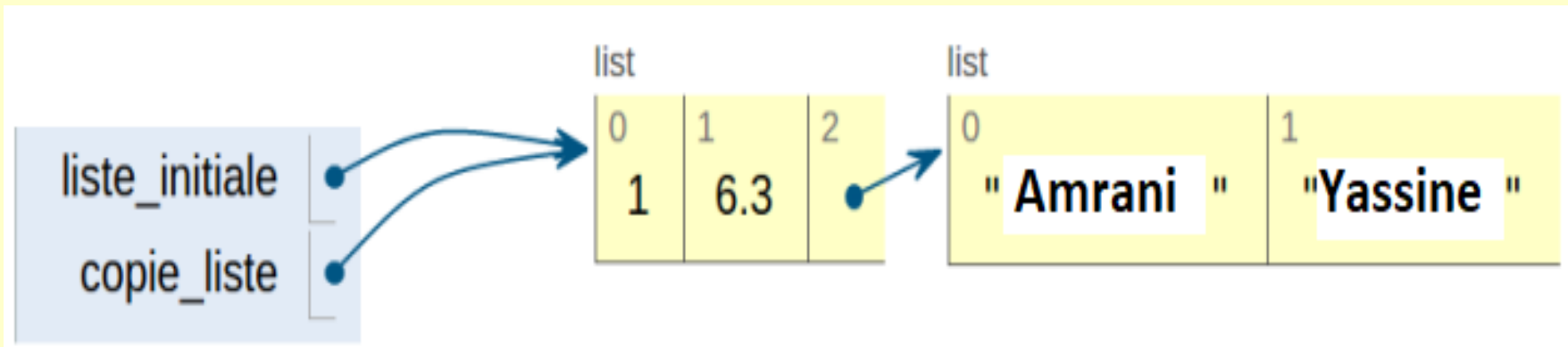
```
liste_initiale = [1, 6.3, ['Amrani', 'Yassine']]
```

```
copie_liste = liste_initiale
```

```
print(copie_liste)
```

Résultat : [1, 6.3, ['Amrani', 'Yassine']]

- Mais en faisant cela, nous avons simplement créé un alias, c'est-à-dire que nous avons instancié un nouvel objet qui partage la même référence que la liste copiée! Autrement dit nous avons donné un deuxième nom **copie_liste** de la liste **liste_initiale**.



```
liste_initiale = [1, 6.3, ['Amrani', 'Yassine']]
```

```
copie_liste = liste_initiale
```

```
liste_initiale[0]=4
```

```
print(liste_initiale)
```

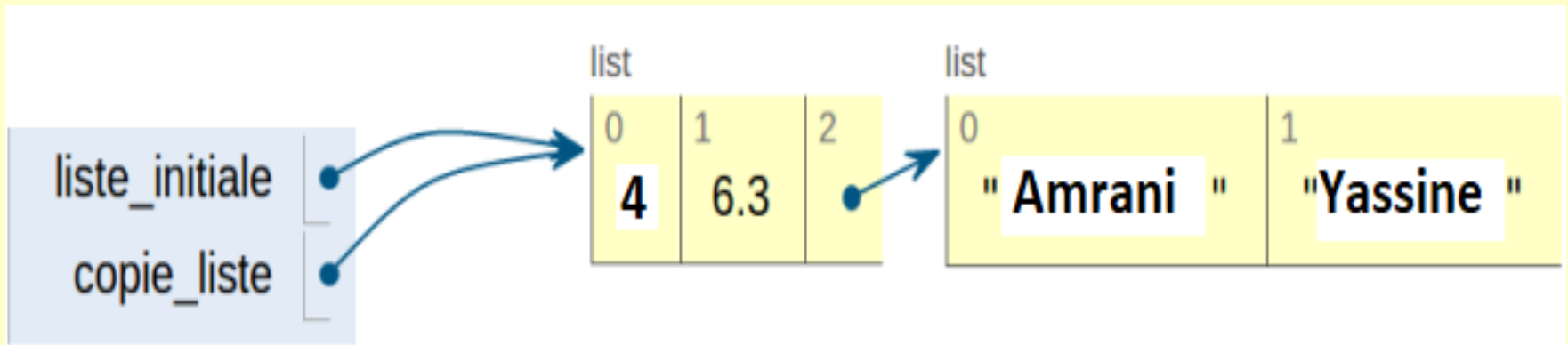
```
print(copie_liste)
```



```
[4, 6.3, ['Amrani', 'Yassine']]
```

```
[4, 6.3, ['Amrani', 'Yassine']]
```

On remarque clairement la dépendance entre la liste et sa copie.



Faire une shallow copy avec list() ou liste[:]

Essayons de voir si nous pouvons solutionner le problème en déclarant une nouvelle liste et en lui affectant **liste_initiale**. Deux possibilités s'offrent à nous :

```
liste_initiale = [1, 6.3, ['Amrani', 'Yassine']]
```

```
copie_liste = list(liste_initiale)
```

Ou bien

```
copie_liste = liste_initiale[:]
```

Le schéma suivant résume la situation



```
liste_initiale = [1, 6.3, ['Amrani', 'Yassine']]  
copie_liste1 = list(liste_initiale)  
copie_liste2 = liste_initiale[:]  
liste_initiale[0]=4  
liste_initiale[2][0]='Saadi'  
liste_initiale[2][1]='Anis'  
print(liste_initiale); print(copie_liste1); print(copie_liste2)
```

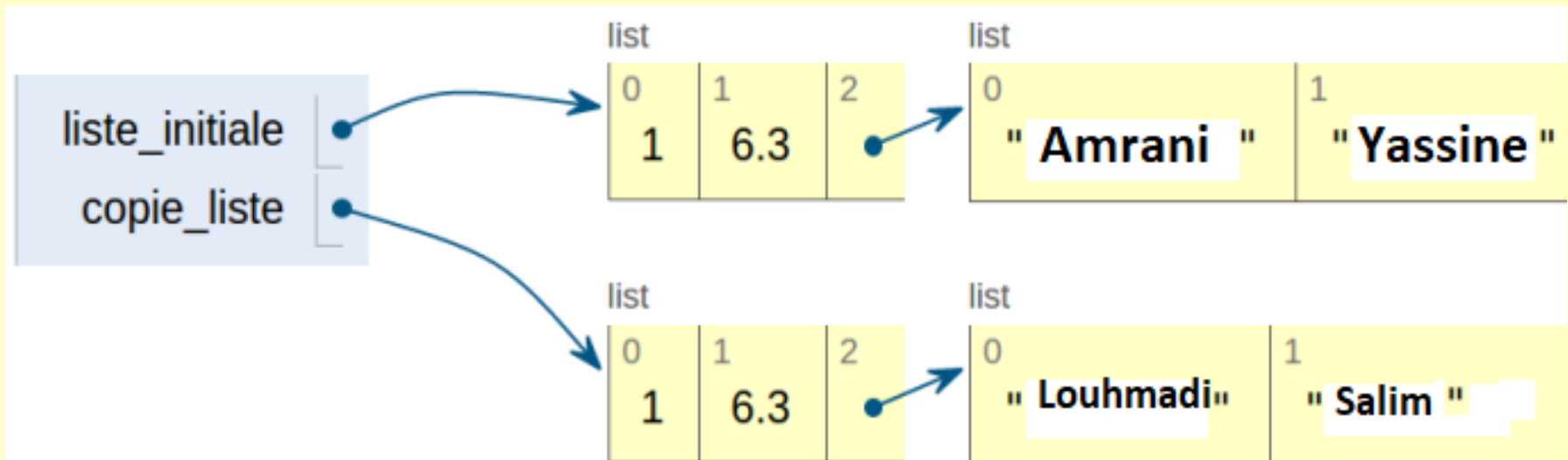


```
[4, 6.3, ['Saadi', 'Anis']]  
[1, 6.3, ['Saadi', 'Anis']]  
[1, 6.3, ['Saadi', 'Anis']]
```

Avec le code ci-dessous, on arrive certes à faire une copie profonde mais ce n'est pas l'idéal.

```
liste_initiale = [1, 6.3, ['Amrani', 'Yassine']]  
copie_liste = list(liste_initiale); copie_liste[2] = list(liste_initiale[2])  
liste_initiale[2][0]='Louhmadi'; liste_initiale[2][1]= 'Salim'  
print(liste_initiale ); print(copie_liste)
```

➔ [1, 6.3, ['Louhmadi', 'Salim']]
[1, 6.3, ['Amrani', 'Yassine']]



Dans le cas d'une liste représentant un arbre (liste de listes de listes de listes) la solution présentée en haut n'est pas très pratique!

Le module **copy** qui fonctionne avec tous les types mutables, permet de faire deux sortes de copies différentes:

- la **shallow copy** (*shallow* signifie superficiel ou peu profond.)
- la **deep copy** (*deep* signifie profond.)

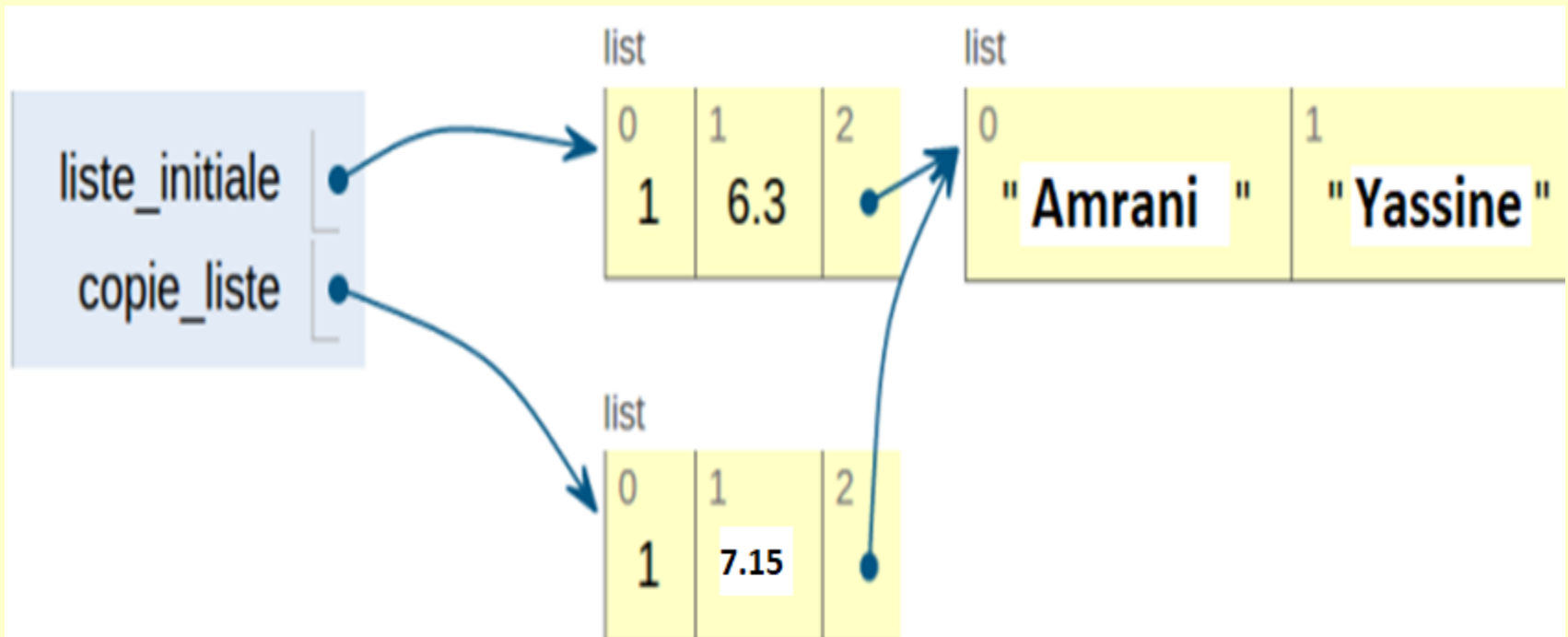
Pour utiliser ces deux méthodes, il faut importer le module copy:

- **copy.copy** effectue une shallow copy
- **copy.deepcopy** effectue une deep copy
- une shallow copy effectue une copie des éléments de premier niveau. Elle copie les éléments **1** et **6.3** tandis que le troisième élément (qui est une liste) devient une référence partagée. Cela signifie que si vous remplacez « **Amrani** » par « **Saadi** » dans la petite liste qui se trouve à l'indice 2 de la liste `copie_liste`, cette modification affectera **liste_initiale** car ce n'est pas un élément de premier niveau.

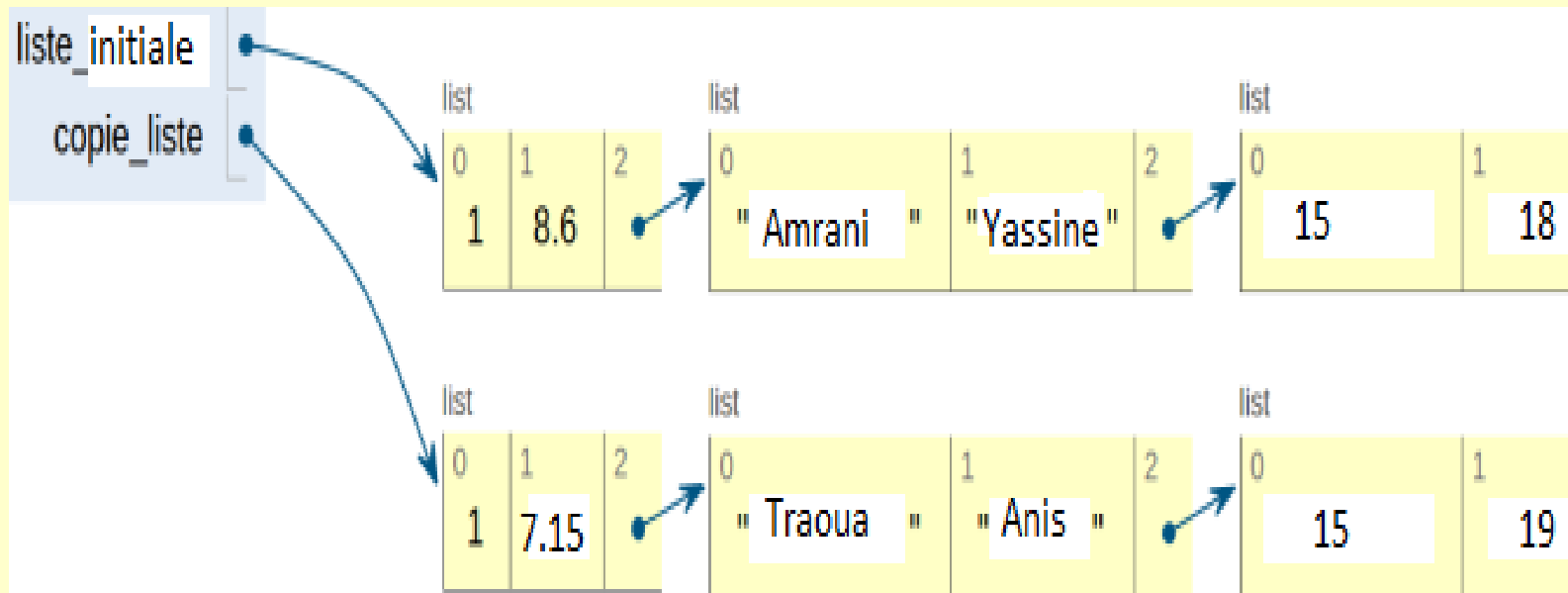
```
import copy
liste_initiale = [1, 6.3, ['Amrani', 'Yassine']]
copie_liste = copy.copy(liste_initiale)
liste_initiale[1]=7.15 ; liste_initiale[2][0]='Saadi';
    liste_initiale[2][1]='Anis'
print(liste_initiale); print(copie_liste)
```

➔ [1, 7.15, ['Saadi', 'Anis']]

[1, 6.3, ['Saadi', 'Anis']]



```
import copy
liste_initiale = [1, 6.3, ['Amrani', 'Yassine'], [15, 18]]
copie_liste = copy.deepcopy(liste_initiale)
liste_initiale[1]=7.15 ; liste_initiale[2][0]='Saadi';
    liste_initiale[2][1]='Anis' ; liste_initiale[2][2][1]=19
print(liste_initiale); print(copie_liste)
➔ [1, 7.15, ['Saadi', 'Anis', [15, 19]]]
    [1, 6.3, ['Amrani', 'Yassine', [15, 18]]]
```

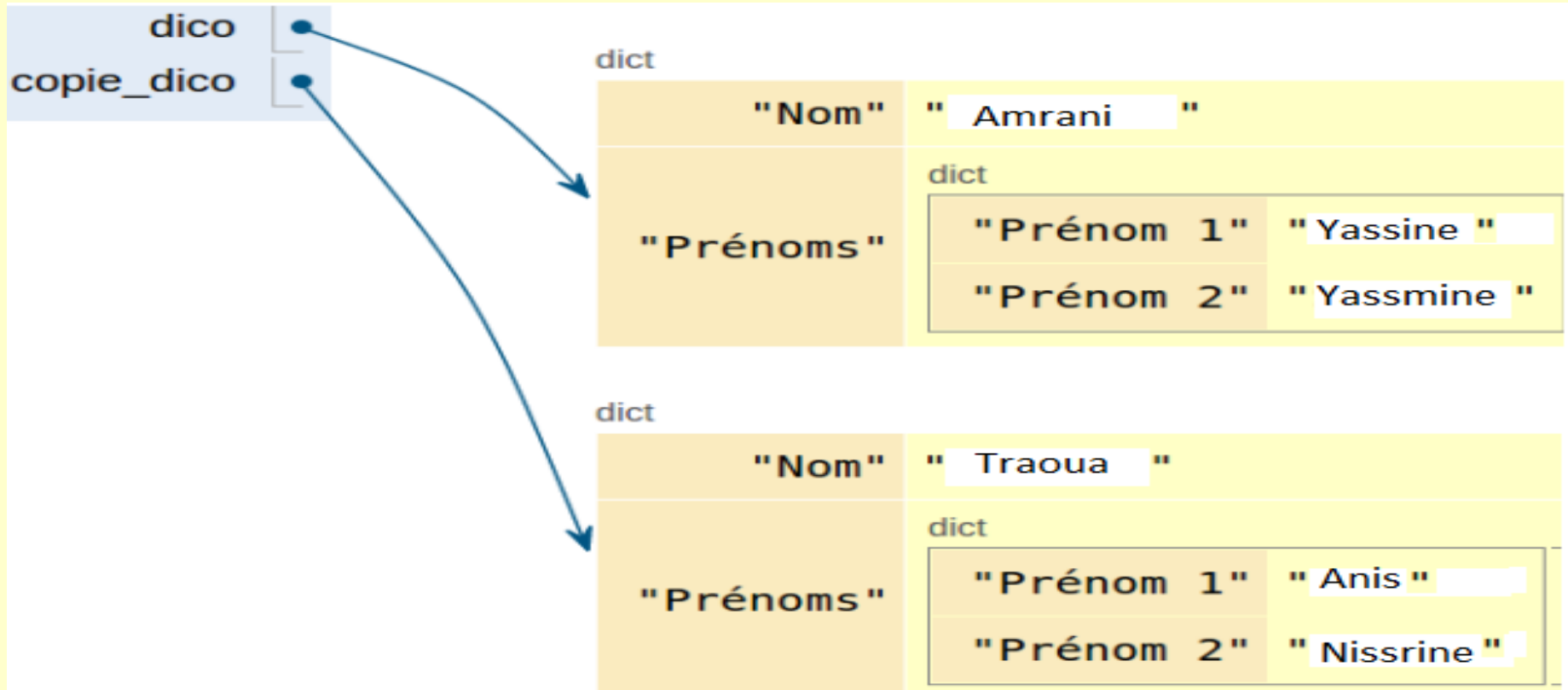


Le module **copy** (avec ses méthodes **shallow copy** et **deep copy**) fonctionne parfaitement avec d'autres objets mutables tels que les dictionnaires.

```
import copy  
  
dico = {"Nom" : "Amrani", 'Prénoms' : {'Prénom 1' : "Yassine",  
    'Prénom 2' : "Yasmine"}}  
  
copie_dico = copy.deepcopy(dico)  
copie_dico['Nom'] = "Saadi"  
copie_dico['Prénoms']['Prénom 1'] = "Nissrine"  
print(dico) ; print(copie_dico)
```



```
{'Nom': 'Amrani', 'Prénoms': {'Prénom 1': 'Yassine', 'Prénom 2':  
    'Yasmine'}}  
  
{'Nom': 'Saadi', 'Prénoms': {'Prénom 1': 'Nissrine', 'Prénom 2':  
    'Yasmine'}}
```



Le même raisonnement s'applique sur les tuples:

```
T1=(1,2,3,[4,6,[7,8,9]]) ; T2=T1; T3=copy.deepcopy(T1)
```

```
T1[3][0]=7777; print(T1); print(T2);print(T3)
```

➔ (1, 2, 3, [7777, 6, [7, 8, 9]])

(1, 2, 3, [7777, 6, [7, 8, 9]])

(1, 2, 3, [4, 6, [7, 8, 9]])

Le même raisonnement s'applique sur les ensembles

```
import copy
```

```
S1=set(); S1.add(10); S1.add('Master DSBD');S1.update({11,12,13})
```

```
S2=S1 ; S3=copy.deepcopy(S1)
```

```
S2.remove(10) ; S3.remove(12); print(S1); print(S2); print(S3)
```



```
{'Master DSBD', 11, 12, 13}
```

```
{'Master DSBD', 11, 12, 13}
```

```
{'Master DSBD', 10, 11, 13}
```

Avec les objets de type **str()**, **int()** ou bien **float()**, il est inutile d'utiliser le module **copy** puisque de toute façon, l'affectation va créer un nouvel objet qui aura sa propre référence.